



18th Iranian Statistics Conference
K.N.Toosie University of Thechnology

29-31 August 2026



Graphical Finite-Population Sampling with Adaptive Monte Carlo Tree Search

Bardia Panahbehagh¹, Mehdi Mohebbi²

¹Faculty of Mathematical Sciences and Computer, Kharazmi University, Tehran, Iran

²School of Mathematics, Statistics and Computer Science, University of Tehran, Tehran, Iran

Abstract:

Graphical finite-population sampling represents first-order inclusion probabilities as bars on a unit-height graph. Moving bar segments preserves the prescribed inclusion probabilities, while changing the second-order inclusion probabilities and hence the variance of the Narain–Horvitz–Thompson estimator. This paper presents a short conference version of this framework and studies an adaptive multi-start Monte Carlo tree search for finding lower-variance graphical sampling designs. Numerical results from a tuning experiment show that the method is sensitive to restart and rollout parameters, and that an aggressive restart rule substantially improves the best design found.

Keywords: Finite-population sampling, graphical sampling, inclusion probabilities, Monte Carlo tree search, Narain–Horvitz–Thompson estimator.

Mathematics Subject Classification (2010): 62D05, 62K05, 90C59.

1 Introduction

Let $\mathcal{U} = \{1, \dots, N\}$ be a finite population, and let $\pi_k = \Pr(k \in S)$ be prescribed first-order inclusion probabilities with $0 < \pi_k < 1$ and $\sum_{k \in \mathcal{U}} \pi_k = E(n_S)$. For a study variable y , the Narain–Horvitz–

¹Corresponding Author: bardia.panahbehagh@khu.ac.ir

Thompson estimator of the population total $Y = \sum_{k \in \mathcal{U}} y_k$ is

$$\hat{Y}_{\text{NHT}} = \sum_{k \in S} \frac{y_k}{\pi_k}.$$

This estimator is unbiased when the first-order inclusion probabilities are respected. However, its variance depends on the full sampling design through the second-order inclusion probabilities. Therefore, after fixing the π_k 's, an important problem is to construct a feasible design whose second-order structure gives a small value of $\text{Var}(\hat{Y}_{\text{NHT}})$.

Graphical finite-population sampling, introduced by Panahbehagh (2026), provides a visual and constructive way to address this problem. Each first-order inclusion probability is represented by a vertical bar of length π_k in the unit interval. A random horizontal line determines the selected units. Rearranging the bars keeps the first-order probabilities unchanged, but modifies the joint probabilities. This makes graphical sampling a natural state space for intelligent search algorithms.

The aim of this paper is to combine graphical sampling with the adaptive multi-start Monte Carlo tree search (MCTS) implemented in the `graphical-sampling` package. The algorithm is designed to improve an initial graphical design according to an exact variance criterion. We summarize the method and report a compact simulation study based on the tuning results for the MCTS implementation.

2 Graphical Finite-Population Sampling

In graphical finite-population sampling, a design is represented on a two-dimensional graph. The horizontal axis contains the units $1, \dots, N$, and the vertical axis is the interval $[0, 1]$. For each unit k , a bar of length π_k is placed somewhere in $[0, 1]$. A random value $r \sim U(0, 1)$ is drawn, and the sample consists of the units whose bars intersect the horizontal line at height r .

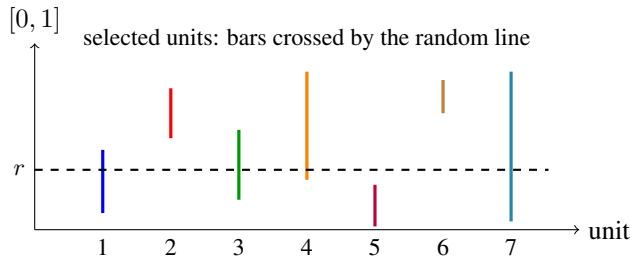


Figure 1: A graphical sampling design. Each bar has length π_k . The random line selects the units whose bars it crosses. Moving bar segments preserves first-order inclusion probabilities but changes the joint selection structure.

The preservation of first-order inclusion probabilities follows immediately: the probability that the random line crosses the bar of unit k is exactly the length of the bar, namely π_k . If the vertical axis is partitioned

into strips with heights $p_1, \dots, p_T$, each strip corresponds to a sample s_t and therefore to a sampling design

$$p = \{p_t = \Pr(S = s_t) : t = 1, \dots, T\}.$$

The second-order inclusion probabilities are then

$$\pi_{k\ell} = \sum_{t: k, \ell \in s_t} p_t, \quad k, \ell \in \mathcal{U}.$$

For the study variable y , the exact design variance can be written as

$$\text{Var}(\hat{Y}_{\text{NHT}}) = \sum_{k \in \mathcal{U}} \sum_{\ell \in \mathcal{U}} \frac{y_k y_\ell}{\pi_k \pi_\ell} (\pi_{k\ell} - \pi_k \pi_\ell). \quad (2.1)$$

Thus, within the same vector of inclusion probabilities, different graphical arrangements can have different precision.

The key operation is to select two strips and move a segment of a bar from one strip to another vacant position. If the segment length is controlled by a parameter α , this local move keeps the total bar length for every unit fixed. Consequently, it preserves the first-order inclusion probabilities while changing the samples and their probabilities. Iterating this operation produces a large family of feasible designs, including fixed-size designs when the move is restricted to configurations that preserve sample size.

3 Adaptive Multi-Start MCTS

The optimization problem is to find, among feasible graphical designs with the prescribed π , a design with small value of the variance criterion in (2.1). The implementation studied here uses an adaptive multi-start MCTS. Each node in the tree stores a graphical design. The reward of a node is based on the improvement in the best variance value observed during a rollout.

The tree begins with a virtual root whose children are the initial designs. At each iteration, the algorithm selects children using the usual upper confidence principle

$$\bar{R}_j + C \sqrt{\frac{\log N_j}{n_j}},$$

where $\bar{R}_j$ is the average reward of child j , n_j is its number of visits, N_j is the number of visits to the parent, and C is the exploration constant. Expansion creates new children by applying graphical design mutations. The number of mutations is split between high, middle, and low magnitudes, with the distribution depending on the previous quality of the node.

Algorithm 1. *Adaptive multi-start MCTS for graphical sampling designs.*

- *Start from one or more valid graphical designs and compute the initial variance criterion.*

- *Select a tree node by the UCB rule and expand it using high-, mid-, and low-magnitude graphical mutations.*
- *For each new child, run an adaptive rollout. At every rollout step, try an aggressive mutation and, if needed, a conservative mutation; retain the better candidate.*
- *Stop a rollout early if its current criterion becomes more than 15% worse than the initial criterion.*
- *Back-propagate the reward $100(C_0 - C_{\text{best}})/C_0$ when an improvement is found, and 0 otherwise.*
- *If the search stagnates, rebuild the tree from the best design found so far.*

This structure is useful for graphical sampling because the design space is large and local moves can easily become trapped. The multi-start root gives the search several initial directions, rollouts provide short-term local improvement, and the restart mechanism preserves the best design while escaping unproductive branches.

4 Simulation Results

The simulation report studies the sensitivity of MCTS tuning parameters while all other settings are fixed. The criterion is the exact NHT variance of the current graphical sampling design, so smaller values are better. Unless a parameter is being varied, the baseline settings are 150 iterations, population size 10, eight children per node, 20 base changes, switch coefficient 1.0, exploration constant 10, and 18 rollout steps. The main findings are summarized in Table 1. The values shown are the best criterion values reported among repeated runs for each tested setting.

Table 1: Summary of the MCTS tuning experiment. Smaller criterion values are better.

Parameter	Tested settings	Best setting	Best criterion
Maximum children per node	5, 8, 10, 15, 20	8	163103.34
Base changes	5, 10, 15, 20, 25	20	163103.34
Switch coefficient	0.5, 0.7, 0.9, 1.0	1.0	163103.34
Exploration constant	0.5, 1.5, 5, 10, 15	10	163103.34
Rollout steps	10, 18, 30	30	162093.85
Population size	1, 10, 20, 30, 40	10	163103.34
Early stopping factor	1.15, 1.30, 1.50, 1.70	1.15	163103.34
Restart after no improvement	3, 4, 5, 8	3	131905.23

The most important result is the effect of the restart parameter. When the tree was rebuilt after three non-improving iterations, the best criterion decreased to 131905.23, compared with 163103.34 under the less aggressive baseline restart. This indicates that stagnation is a major issue in the graphical-design search space, and that frequent restarts from the best current design can be more valuable than continuing to expand a stale tree.

The number of children per node also matters. A small value gives too little breadth, while very large values increase computation without improving the best criterion. In this experiment, 8 children per node gave the best balance. The parameter `base_changes` = 20 was also a good setting, although nearby values produced similar results, suggesting that the adaptive rollout can compensate for moderate changes in mutation size.

For the switch coefficient, the results favor values near 1.0; values 0.5 and 0.7 were weaker and often slower. The exploration constant 10 gave the best reported variance among the tested values, supporting a relatively exploratory tree policy. Finally, increasing the rollout length from 18 to 30 gave one slightly better criterion value, but at a clear computational cost. Thus, 18 rollout steps is a practical compromise, while 30 can be used when extra computing time is available.

Discussion and Conclusion

Graphical sampling provides a flexible representation of finite-population sampling designs with fixed first-order inclusion probabilities. The adaptive multi-start MCTS exploits this representation by searching directly over feasible graphical designs and using exact NHT variance as the criterion. The tuning study shows that the method can substantially reduce the criterion, especially when frequent restarts are used to avoid stagnation. Future work should compare this MCTS version with A* search, cube sampling, and determinantal sampling on larger simulated and real populations.

References

- Browne C.B., Powley E., Whitehouse D., Lucas S.M., Cowling P.I., Rohlfshagen P., Tavener S., Perez D., Samothrakis S. and Colton S. (2012), A survey of Monte Carlo tree search methods, *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1), 1-43.
- Deville J.-C. and Tille Y. (2004), Efficient balanced sampling: The cube method, *Biometrika*, 91, 893-912.

- Horvitz D.G. and Thompson D.J. (1952), A generalization of sampling without replacement from a finite universe, *Journal of the American Statistical Association*, 47, 663-685.
- Kocsis L. and Szepesvari C. (2006), Bandit based Monte-Carlo planning, *Proceedings of the 17th European Conference on Machine Learning*, 282-293.
- Loonis V. and Mary X. (2019), Determinantal sampling designs, *Journal of Statistical Planning and Inference*, 199, 60-88.
- Narain R.D. (1951), On sampling without replacement with varying probabilities, *Journal of the Indian Society of Agricultural Statistics*, 3, 169-174.
- Panahbehagh B. (2026), Graphical finite population sampling, *Survey Methodology*, 52(1), 129-150.
- Panahbehagh B., Mohebbi M., HosseiniNasab A.M. and Moghadam M.H. (2025), *graphical-sampling: A Python package for graphical finite-population sampling*, Python package.